

Research-Agent

Installation Guide for GitHub Research Repositories

Set up a research GPT, install reusable Skills, bootstrap a GitHub repository, and keep the toolchain versioned without mixing project state with reusable workflow code.

RESEARCH_AGENT.md Skill source release tooling	persistent Agent instructions installed skill.zip packages	AGENTS.md roadmap + Tasks evidence + history
Reusable behavior	Runtime tooling	Project authority + state

The rule to remember

The Agent and Skills normally live in ChatGPT. Your GitHub repository stores project-local rules and durable research state. Do not copy the Skill source into every project unless you deliberately want a vendored snapshot.

What this guide covers

1	Prepare	Connect ChatGPT to the GitHub repository and choose one Research-Agent version.
2	Install	Configure RESEARCH_AGENT.md and install each Skill independently as skill.zip.
3	Bootstrap	Ask bootstrap-research-project to create the local research system.
4	Research	Work in bounded task conversations; invoke standardized procedures in natural language.
5	Upgrade	Test new Research-Agent releases and update the project lock intentionally.

Prerequisites: a GitHub repository, ChatGPT with GitHub access, and permission to install/configure the Research-Agent tooling you intend to use.

1. Install the Agent and Skills

Choose one Research-Agent release or exact source commit. Keep the persistent Agent instructions and all installed Skills from that same version so the runtime is reproducible.

A. Connect the research repository

Create or choose the GitHub repository that will hold your research. Connect ChatGPT to GitHub and grant that repository read access, plus write access if you want the Agent to publish branches, task files, evidence, or pull requests.

B. Configure collaborator behavior

Use **RESEARCH_AGENT.md** from the chosen Research-Agent version as the persistent instructions for the GPT/agent that conducts research. Do not use the Research-Agent source repository's root **AGENTS.md** as the research prompt; your project gets its own local **AGENTS.md** during bootstrap.

C. Install the core Skills

For the complete workflow, install these independent packages in ChatGPT:

bootstrap-research-project	define-research-task
transcribe-research-evidence	review-mathematical-result
restructure-research-roadmap	complete-research-task
validate-research-project	synthesize-research-project

Each Skill is separate

Install each /skill.zip independently. A Research-Agent release bundle may contain many Skill archives for transport, but the outer bundle is not itself a Skill.

D. If no release bundle is available

```
git clone https://github.com/AidanRDonahue/Research-Agent.git
cd Research-Agent
git checkout <EXACT-COMMIT-OR-TAG>
python scripts/package_skills.py --output dist
```

Install the individual packages under **dist/<skill-name>/skill.zip**. Keep **dist/research-agent-distribution.json**; it records the exact version and source commit used to build the packages.

2. Bootstrap the GitHub Research Repository

Once the Agent and Skills are available in ChatGPT, initialize the target repository around one bounded root research question.

Bootstrap prompt

```
Initialize OWNER/REPOSITORY as a Research-Agent project around the question:
"<ROOT RESEARCH QUESTION>"

Use the existing material in <PATH> as supplied background.
Do not invent research conclusions or future tasks.
```

The bootstrap workflow inspects the repository first, preserves compatible existing material, and creates project-local authorities and research state. The root task starts pending; bootstrap organizes the research but does not solve it.

Typical project structure

```
README.md
AGENTS.md # local operating authority
dictionary.md # terminology / notation
roadmap.yaml # canonical structured roadmap
ROADMAP.md # human-readable projection
research-agent.lock.json # toolchain pin when metadata is available
Tasks/ # task contracts, evidence, resolutions
Background/ # genuinely shared foundations
templates/
history/
checks/
```

Three files that are easy to confuse

RESEARCH_AGENT.md	Research-Agent distribution / agent config	How the collaborator behaves
AGENTS.md	Your research repository	Local project rules and authority
research-agent.lock.json	Your research repository	Which external toolchain version the project expects

Authority rule

The project's current AGENTS.md governs project-local behavior. A reusable Skill supplies procedure and the lock supplies compatibility metadata; neither overrides project authority.

3. Use the Workflow Day to Day

Ordinary research is conversational. Name the current task and the bounded question; the Agent reads the repository and stops at meaningful decision points instead of autonomously finishing the task.

Bounded research prompt

Work with me on T002. For this turn, determine whether the current assumptions are sufficient for the proposed bound. Do not continue to later proof steps.

Standardized procedures

Define	Define a new task under T002 for this obstruction.
Review	Review the current lemma adversarially.
Evidence	Transcribe the verified proof as evidence for T002.
Restructure	Make this obstruction a sibling task and preserve history.
Complete	Evaluate T002 for completion and complete it only if supported.
Validate	Validate this branch against the current project rules.
Synthesize	Synthesize the completed tasks into a paper-level outline.

Research stays user-directed
Defining a task does not authorize solving it. Saving evidence does not imply completion. Completion is a separate explicit workflow, and negative or inconclusive outcomes can be legitimate results when supported.

Repository writes

For substantive changes, expect a scoped branch, the smallest coherent mutation, validation, and a reviewable pull request unless your project explicitly authorizes another write path.

4. Upgrade Safely

Do not make a research project follow Research-Agent/main automatically. Treat a newer release as a compatibility event that must be reviewed and intentionally accepted.

1	Inspect	Read the release changes and identify Agent/Skill contracts that changed.
2	Install	Update the relevant Agent instructions and Skill packages in ChatGPT.
3	Test	Exercise representative workflows against a project branch.
4	Compare	Compare the new distribution with research-agent.lock.json.
5	Reconcile	Make any project-local changes required by the new workflow.
6	Accept	Update the lock only after intentionally accepting the new release.

Compatibility check

```
python scripts/check_project_compatibility.py \  
/path/to/project/research-agent.lock.json \  
--distribution /path/to/research-agent-distribution.json
```

A mismatch is a review signal

A version or commit mismatch does not authorize an automatic migration. The project lock records the external toolchain a project expects; project-local AGENTS.md and current research state remain authoritative.

Quick-start checklist

[]	GitHub research repository exists and ChatGPT can access it.
[]	RESEARCH_AGENT.md is configured as persistent collaborator behavior.
[]	Required Skills are installed independently from one version.
[]	The repository has been bootstrapped around one root question.
[]	Project-local AGENTS.md is treated as the operating authority.
[]	The toolchain is pinned when concrete distribution metadata is available.
[]	Research proceeds through bounded, user-directed task conversations.

Canonical source: AidanRDonahue/Research-Agent. For the full walkthrough, see USING_IN_YOUR_PROJECT.md in the repository.